CASE STUDY | LEGACY SAS → CDP MIGRATION

Rebuilding an Undocumented Analytics Pipeline on a Modern CDP

How a 475-variable marketing-analytics engine was reverse-engineered, migrated and proven correct to 100% variable population — without rewriting the client's code

THE CLIENT

"Meridian Media" — a hypothetical multi-channel publisher selling magazines, books and continuity programmes, and running direct-mail fundraising. Name and figures are illustrative.

Client Analytics · Showcase Edition

The Engagement: Keep Scoring Running Through a Platform Move

Meridian was moving its marketing data from a legacy Oracle warehouse to a cloud CDP. Every campaign model depended on a SAS feature engine nobody had documented — and it had to keep producing the same answers on the new platform

WHAT IT WAS SCOPED AS

A short, time-and-materials remapping: point existing SAS jobs at the new CDP and swap table and field names

WHAT IT BECAME

A discovery and modernisation programme: reverse-engineer the engine, map variable lineage, absorb platform differences and prove parity — while live campaigns kept running

No documentation

Inherited from a prior vendor with no record of embedded logic, derivations or intended execution order

Interdependent code

Driver, orchestrator, definitions, SQL generator and variable recipes — an error could originate in any of them

Data that changed shape

Columns renamed, re-typed or left empty in migration; some legacy objects had several candidate equivalents, others none

Platform behaviour changes

Temp tables, NULL handling and date types behave differently on the CDP than on Oracle — silently, not with errors

WHY THE WORK WAS MORE COMPLEX THAN EXPECTED

The code had to be understood before it could be migrated — discovery, not replacement, drove the timeline

DISCOVERY · READING THE CODEBASE

Deriving the Data Flow From the Code Itself

No design document existed. The architecture was reconstructed by parsing the SAS itself — the include chain, the macro call graph and the SQL the engine actually generated — before a single line was migrated

Parse the include chain 01

Five files load in a fixed order. Establishing it revealed which layer owned connections, lookups, SQL and variable logic.

Map the macro call graph 02

21 SQL-builder macros, 16 variable categories, 500+ recipe calls — each traced back to the builder that emits its query.

Capture the emitted SQL 03

The engine writes every query to its log. Reading emitted SQL — not source — showed what really reached the database.

Rebuild the lineage 04

Each variable traced to source table, filter slot and join key: the dependency map the system had never had.

WHAT THE RECONSTRUCTION EXPOSED

SQL is written at run time

Queries are assembled by shared builders during execution, so a symptom in one variable almost never originates in that variable's code.

Filters live in numbered slots

Every recipe passes conditions into positional macro arguments. Exclusion and sign slots are shared branches serving over a hundred variables.

Dates are stored as offsets

Date variables hold days-since-build, not calendar dates — which changes how any comparison against legacy has to be done.

OUTCOME

The pipeline became legible: files, tiers, 475 output variables and their full lineage documented — the prerequisite that made every later fix targeted rather than speculative

Architecture Overview — End-to-End Flow

As found: a caller configures the run, an orchestrator loads three library files, and the SQL engine round-trips against the source database once per variable to build one wide customer table



WHAT THE DIAGRAM MAKES VISIBLE

The caller only configures; the orchestrator only wires; every query is written at run time by the library — so one corrected pattern in a shared builder propagates to every variable it serves

Before vs After Architecture

The macro framework is unchanged — deliberately. What changed sits at the boundary: how the engine connects, what it maps names to, and how it handles values the legacy platform never produced

BEFORE · LEGACY ORACLE

Data platform

Oracle — columns reliably populated, native temp tables, direct pass-through

Column names

Legacy UPPER_SNAKE names referenced directly by every recipe

Filter semantics

A bare ≠ comparison behaved as expected because NULLs effectively never occurred

Date handling

Date columns arrived as DATETIME; the engine's offset arithmetic assumed it

Assurance

A clean execution log was treated as proof the run was correct

AFTER · CLOUD CDP

Data platform

Spark-based CDP over ODBC — two schemas, server-side staging, scoped temp objects

Column names

A bridge layer maps legacy names to CDP names at run time; recipes untouched

Filter semantics

Every not-equal comparison NULL-safe by construction, in every filter slot of the shared builders

Date handling

Source type detected before conversion; multi-field dates resolved explicitly

Assurance

Variable-level census plus legacy reconciliation is the gate; the log confirms completion only

THE DESIGN DECISION BEHIND THIS

The recipes were left as written. Every platform difference was absorbed in a bridge layer instead — which is why a single mapping correction could move dozens of variables at once

ARCHITECTURE · EXECUTION

How One Run Actually Executes

The orchestrator loads the framework, then the engine generates, runs, stages and merges one query per variable. Understanding this loop is what made both the defects and the runtime explainable

CALLER

01

Driver

Build date, audience keycode, output name, variable list

ORCHESTRATION

02

Orchestrator

Assigns ODBC connections, defines executors, loads the library

MACRO ENGINE

03

Library + bridges

Lookups, 21 builders, 16 categories, two bridge files

CDP SOURCE

04

Analytic + core schemas

Order, subscription, marketing-activity, product and promotion objects over ODBC

PER-VARIABLE LOOP · ~2,400 queries per full run

Build query

Filters, date window and code lists assembled into SQL

Execute on CDP

One ODBC round-trip; timing logged

Stage result

Temp table per variable, keyed on customer

Merge to output

Left-joined onto the wide table by customer ID

OUTPUT One row per customer · 475 feature columns

THEN Drop helper columns · QC census · legacy comparison

THE DESIGN CONSEQUENCE THAT MATTERED MOST

Because SQL is generated at run time, one defective pattern in a shared builder multiplies across every variable it serves — and one fix there repairs them all at once

ENABLEMENT · WHERE AI FIT

Where AI Fit in the Migration

Not as a code generator. AI was applied where the bottleneck actually was — reading an undocumented codebase faster than a person could, and turning 60MB+ logs into evidence a human could act on

Comprehension at speed 01

Interdependent files, 500+ macro calls. The include chain and call graph were traced in hours rather than the weeks a manual read would take.

Log forensics 02

Logs ran past 60MB and ~5,000 errors. Every error was classified, counted by type, and each cascade tied back to the single predicate that caused it.

Diagnostic scripting 03

Each hypothesis became a read-only probe — profile a column, test both predicates, compare routes — so conclusions rested on data.

Documentation as a by-product 04

Architecture notes, lineage maps, decision and fix registers were written as the work happened, not retro-fitted.

WHAT AI DID NOT DO

It did not decide. Every mapping, fix and sign-off was a human judgement made against evidence — and twice the confident first explanation was overturned by profiling the source.

WHERE THE LEVERAGE CAME FROM

The constraint was never typing speed — it was understanding a system nobody had documented. AI compressed the comprehension step, which is what let a stalled migration move again.

THE SHAPE OF THE COLLABORATION

AI read, traced, counted and drafted; the team decided, verified and signed off — the division that kept speed from turning into risk

METHOD · HOW WE GOT THERE

Solution Derivation Path

How a reported symptom became a shipped fix. The same path was walked for every defect — and the branch after step 4 separated a code fix from a data escalation

01

SYMPTOM REPORTED

Client flags a variable below the 90% match threshold, or QC surfaces an all-zero column.

Match-rate report · population census

02

CLASSIFY THE FAULT

Absent, zero-filled or understated? Each implies a different test; choosing wrongly wastes the next two steps.

Log scan · value distribution

03

READ THE EMITTED SQL

Pull the generated predicate from the run log. What the engine sent is rarely what the source appears to say.

Run log · macro trace

04

PROFILE THE SOURCE

Query the CDP directly — is the column populated, do the keys share a domain, do the filter values exist?

Read-only probe

STEP 05 · THE BRANCH THAT DECIDES EVERYTHING AFTER IT

SOURCE HAS THE DATA → CODE FIX

Correct the shared generator, never the individual variable. Re-run on the small test audience, confirm by value — not by absence of errors — and only then promote to the full population.

SOURCE IS EMPTY → DATA ESCALATION

Evidence the gap at source, confirm the variable populates in legacy, and hand it to the platform team as a data-availability item with a proposed resolution and an owner.

WHY THE PATH MATTERED MORE THAN ANY SINGLE FIX

Walked end-to-end, it turned three proposed rebuilds into one-line corrections and routed true data gaps to the team that could close them — the route that ended at 475 of 475

METHOD · DIAGNOSE BEFORE YOU FIX

From Symptom to Root Cause to Fix

Variables failing silently cannot be fixed by reading code. Every defect went through the same four steps — and step three repeatedly overturned the obvious explanation

01 Classify the symptom

Absent, all-zero or populated-but-wrong are three different faults, each needing a different test.

02 Isolate the emitted SQL

Read what the engine actually sent, not what the source appears to say. The emitted predicate is the evidence.

03 Profile the source column

Is it populated, is the join key shared, do the values exist? This step overturned two confident diagnoses.

04 Fix the generator

Correct the shared macro, not the variable — then confirm by value, not by absence of errors.

WORKED EXAMPLE · 27 AFFINITY VARIABLES AT ZERO

THE ASSUMPTION

One product code spans titles with different content classes, so the join resolves ambiguously.

THE PROFILE

The mapped product-code column held a NULL on every single row — zero distinct values.

THE ACTUAL CAUSE

The join could not match anything. A differently named product-code column was fully populated and matched billions of rows.

THE FIX

One line in the column mapper. All 27 variables recovered — no lookup table, parse rule or key change needed.

THE PRINCIPLE

Profile the source before changing the code. Every proposed remedy for this defect was a rebuild; the real fault was a mapping pointed at a column containing no data

Three Defects, Three Different Failure Modes

Each arrived as "the variable isn't right". Diagnosis showed three unrelated causes — and none of them announced itself clearly

Silent NULL exclusion

SYMPTOM

Revenue variables populated, but understated. No error, no warning.

ROOT CAUSE

Under ANSI SQL a NULL compared with ≠ evaluates to UNKNOWN, so those rows vanished. Oracle never exposed this — the columns were always populated there.

(col is null or col ≠ val)

✓ **~35% of sampled magazine revenue restored**

Empty column mapping

SYMPTOM

27 affinity variables at zero for every customer.

ROOT CAUSE

The bridge mapped a legacy product code to a CDP column that migrated 100% NULL, so the join matched nothing at all.

legacy code → populated CDP column

✓ **27 variables recovered by a single line**

Wrong source field

SYMPTOM

Email-tenure date present, but every value recent.

ROOT CAUSE

The advised CDP field records when the record was loaded, not when the address was captured — so migrated records all carried the migration date.

coalesce(created date, load timestamp)

✓ **Original capture dates back a decade restored**

THE COMMON THREAD

Every one traced to a legacy assumption that no longer held — a column always populated, a name carrying its data forward, a field meaning what it used to mean

ASSURANCE · QC WORKFLOW

Self-Audit First, Then Validate Against Legacy

A clean log proves the code ran, not that the output is right. Every run passes an internal audit before any comparison with the legacy platform is attempted

STAGE 1 · SELF-AUDIT (inside the CDP)

Row-count integrity

Output must equal the requested audience exactly. A higher count means a merge appended rows instead of joining.

Variable population census

Every column classified populating, all-zero or all-blank. The zero list is the working defect queue.

Attribute comparison

Type, length and format checked against the legacy structure so differences surface as cosmetic or real.

Source reconciliation

Suspect variables re-derived directly against the CDP to confirm what the engine should have returned.

STAGE 2 · LEGACY VALIDATION

Normalise before comparing

Build dates differ and dates are stored as offsets — both sides converted to calendar dates first.

Match on shared customers only

The two universes are not identical; comparing on the overlap avoids false mismatches.

Band the results

Variables grouped by match rate against the agreed 90% threshold so attention goes where it matters.

Separate defect from difference

Re-classification and extra months of activity are expected variance — not every gap is a bug.

THE GATE

Nothing was signed off on a clean log. A run with zero errors still carried 31 all-zero variables — the census, not the log, is what caught them

PRACTICE · ITERATION SPEED

Building a Test Harness That Told the Truth

A full run took about an hour, which made verification — not repair — the bottleneck. A deliberately built small audience reproduced the full failure profile at a fraction of the cost

~600K
Customers in the production audience

~1K
Customers in the test harness

Identical
Error profile reproduced by the harness

1 / 570
Of the data volume

HOW THE AUDIENCE WAS CHOSEN

Customers were ranked by how many variables actually carried data — not just non-missing, because the engine zero-fills unmatched numerics.

The richest records exercised nearly every recipe, and a score threshold rather than a fixed count kept tied customers from being dropped arbitrarily. The audience was registered under its own isolated keycode.

WHAT IT CHANGED

- Same ~5K errors, same type-failure cascade, same 13 missing variables as the full run — a fix proven here was proven everywhere.
- Each change could be verified on its own instead of batched, so cause and effect stayed attributable.
- Production audience and outputs were never touched while debugging.

THE CAVEAT WE KEPT IN VIEW

A harness of the richest records is right for proving code paths and wrong for measuring fill rates — final population was always confirmed on the full ~600K audience, never the sample

Four Failures That Had Nothing to Do With the Variables

Not every broken run was a logic defect. These four produced empty or wrong output with little or no warning — and each would have been mis-diagnosed as a code problem without the run discipline that caught it

01

Leftover staging table

WHAT HAPPENED

An aborted run left its permanent audience table behind. The next run's CREATE failed, the audience loaded empty, and the job finished 'successfully' with zero rows.

HOW IT WAS CAUGHT

Output created with 0 rows; the CREATE error sat in the redirected log

SAFEGUARD

Existence guard drops any stale table before rebuild

02

Key-domain mismatch

WHAT HAPPENED

A new recipe grouped on the platform's profile key but labelled it as the customer key. The merge matched nothing and appended rows — output grew by half.

HOW IT WAS CAUGHT

Output row count exceeded the requested audience by ~50%

SAFEGUARD

Row-count check: output must equal the audience exactly

03

Unseen reference data

WHAT HAPPENED

A new product code landed in the CDP after go-live. A pre-flight integrity check stopped the whole run before a single variable was built.

HOW IT WAS CAUGHT

Integrity-check message naming the unknown code

SAFEGUARD

Code added to a neutral, unread group; family confirmed later

04

Overwritten bridge file

WHAT HAPPENED

A one-macro patch was saved over the full compatibility file. Every mapping macro disappeared and hundreds of variables failed to resolve.

HOW IT WAS CAUGHT

Thousands of 'macro not resolved' warnings; fresh-session check returned 0

SAFEGUARD

Compile check after every deploy: each bridge macro must exist

WHY THIS MATTERS

On a shared production platform, the environment fails as often as the code. Each trap became a permanent guard, so the next team inherits the protection rather than the incident

PRACTICE · PRODUCTION SAFETY

Run Discipline on a Shared Production Platform

All work happened against live production data that campaigns depended on. These rules were applied to every change — they are as much a part of the method as the diagnosis itself

Read-only diagnostics first

Every hypothesis was tested with SELECT-only probes before any code changed. No write ever went to a shared schema to answer a question.

Verify the environment, not just the code

Confirm connection, schema and path before running — test and production shared names, and two code trees existed side by side.

Isolate test output

Test runs wrote under their own keycode and output name. Production datasets were never the target of an experiment.

Back up before overwrite

The engine renames the previous output on failure. The last good dataset was copied to a protected name before every risky run.

Edit the real file, not a reconstruction

Fixes were applied byte-for-byte to the uploaded source, encoding and line endings preserved, then diffed so only the intended lines changed.

Prove the file compiled

After deployment, every macro the bridge should define was checked to exist in a fresh session — before the long run, not after it.

Read the right log

The engine redirects its log mid-run. The visible log often showed only the final symptom; the real error sat in the redirected file.

One change, one verification

Each fix ran on the test harness and was confirmed by value before the next was layered on top.

THE OUTCOME OF THE DISCIPLINE

Dozens of iterations against production with no campaign disruption and no lost output — and every change traceable to the evidence that justified it

PERFORMANCE · WHERE THE TIME GOES

Why a Smaller Audience Did Not Mean a Faster Run

Cutting the audience from ~600K to ~1K customers barely changed the runtime. Timing every query in the log explained why — and redirected optimisation effort to where it would actually pay

~2,400

Queries issued per full run

~5 sec

Fixed overhead per query, regardless of size

~1 hour

Runtime even on the ~1K test audience

57%

Of runtime in queries of 4–6 seconds

WHAT THE TIMING SHOWED

- Every source view averaged ~5 seconds per query, whether it returned a thousand rows or scanned millions.
- That uniform floor is scheduling and round-trip latency, not data volume.
- Runtime therefore scales with the number of variables requested — not with the number of customers.

LEVERS, IN ORDER OF PAYBACK

Subset the variable list for debugging

Validate a fix on the 10–20 variables it touches: minutes, not an hour.

Run variable families in parallel

Independent categories in separate sessions with distinct staging names.

Stage the heaviest views once

Pre-filter the four largest views to the audience, then query the small copy.

THE LESSON FOR SIZING

Measure before optimising. The intuitive lever — a smaller audience — was the wrong one; query count and per-query latency were the real cost drivers

DELIVERY · ITERATION TIMELINE

Seven Iterations From First Run to 100% Population

Each iteration closed one class of defect and was verified by value before the next began. Ownership was split by role so that whoever fixed a defect was never the only one validating it

MONTH 1

Parallel run in production

First CDP execution; unresolved names cascaded across the log.

344/475 · ~5K err

Analytics engineer

MONTH 2

Bridge layer & sample check

Name mapping added; a bad key rewrite removed; reference sample matched.

423 / 475

Engineer · QC lead

M3 · WK1

Variable triage

Sub-90% variables grouped by shared root cause, not chased one by one.

6 families

Engagement lead

M3 · WK2

NULL-safe exclusions

Exclusion predicates made NULL-safe; the type-failure cascade cleared.

~5K → 3 err · 436

Analytics engineer

M3 · WK3

Date source & mapping fix

Date re-sourced; empty product column remapped; affinity recovered.

464 / 475

Analytics engineer

M3 · WK4

Recent build date

Run moved to a current build date; boundary variables populated.

469 / 475

QC lead

MONTH 4

Final-six closure

Sign-slot fix, refreshed product keys, full-audience validation, one port.

475 / 475

All roles

ROLE MODEL

Engagement lead
Client communication · escalation · scope

Analytics engineer
Engine diagnosis · code remediation

QC lead
Census workbooks · legacy reconciliation

WHAT THE CADENCE ENFORCED

No iteration began until the previous one was verified by value. Seven passes took the run from ~5,000 errors and 344 populating variables to zero errors and 475 of 475

OUTCOMES · WHERE THE PIPELINE STANDS

Results After Remediation

Measured on the full ~600K production audience against the 475-variable legacy structure

~5K → 0
Errors in the execution log

475 / 475
Variables populating (100%)

2
Bridge files hold every platform change

~35%
Magazine revenue restored by one fix

WHAT MOVED · 344 → 475

+79	promotion variables	restored by removing a wrong key rewrite
+13	interest variables	recovered by NULL-safe exclusions
+1	date variable	re-sourced from two fields
+27	affinity variables	recovered by one mapping correction
+5	boundary variables	reached by a recent build date
+6	final variables	closed in the last iteration (right)

HOW THE FINAL SIX WERE CLOSED

1 channel variable
NULL-safe fix extended to the sign slots; client confirmed blank channel = non-web.

2 promotion variables
Retired product identifiers replaced by current ones via a client-supplied lookup in the bridge.

3 activity-sparse variables
Populated on the full audience (two) and one legacy variable ported as a new recipe.

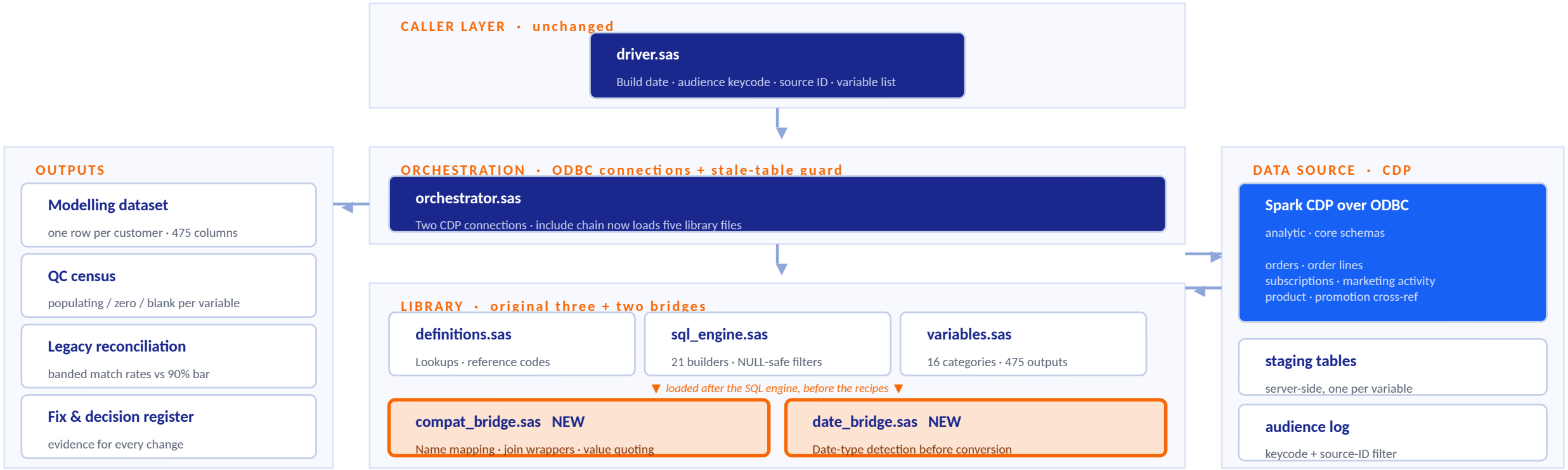
READINESS

The pipeline runs clean end-to-end on the CDP with every one of 475 variables populating, each closure evidenced at source, reconciled against legacy and signed off by the business

ARCHITECTURE · CURRENT STATE

Current Architecture — Seven Files, Two of Them New

The same flow as the original, with two bridge files loaded between the SQL engine and the variable recipes. Nothing upstream or downstream was rewritten — the new files translate, they do not replace



WHY TWO NEW FILES RATHER THAN EDITS TO FIVE

Every platform difference lives in the bridges. The original library stays legacy logic — the client's code remains recognisably theirs, and every CDP accommodation sits in one reviewable place

BRIDGE FILE 1 OF 2

compat_bridge.sas — The Name and Join Translator

Legacy recipes reference column names that no longer exist on the CDP. Rather than rewrite 500+ recipe calls, this file intercepts every name at run time and returns the CDP equivalent

WHY IT WAS CREATED

The problem

The CDP renamed core columns and re-typed others. Recipes sent legacy names that did not exist, and joins failed to resolve — the source of most of the ~5K errors.

The alternative rejected

Editing every recipe would have made the client's code unrecognisable and put each working variable at risk.

How we found it

The emitted SQL in the run log showed literal legacy names reaching the CDP, where no such columns existed.

WHAT IT CONTAINS

Column mapper

Per-table legacy→CDP name lookup. Unknown names pass through unchanged — zero risk to anything not listed.

Join wrappers

Override the engine's join builders so both sides of every join resolve through the mapper.

Value quoting

Numeric-looking values compared with character columns are quoted automatically.

Lookups & recipes

Product-key refresh lookup and a dedicated two-field date recipe live here too.

POST-IMPLEMENTATION EFFECT

+79	promotion variables restored when an incorrect key rewrite was removed
+27	affinity variables recovered when one product-code mapping was corrected
+2	promotion variables closed via a refreshed product-key lookup
0	recipes rewritten to accommodate the platform

THE BRIDGE PRINCIPLE

One file holds every naming difference between the platforms. Correcting a mapping there is a one-line, reviewable change that reaches every variable that depends on it

BRIDGE FILE 2 OF 2

date_bridge.sas — The Date Semantics Guard

The engine's date arithmetic assumed Oracle DATETIME values. The CDP returns several date types across the same columns, so conversion had to become type-aware before it could be trusted

WHY IT WAS CREATED

The problem

Every date variable is computed as build date minus the date part of the value. On the CDP some columns return DATE, so the conversion silently produced a wrong offset.

Why it was invisible

No error is raised. The variable populates with a plausible number — just the wrong one, which only a legacy comparison exposes.

How we found it

A days-since-shipment variable came back carrying the build date itself. The client spotted it in the output before any log did.

WHAT IT CONTAINS

Type detection

Reads the returned type at run time instead of assuming DATETIME, choosing the right conversion per column.

Guarded conversion

Date-part extraction only where valid; DATE columns are differenced directly against the build date.

Override, not edit

Loaded after the engine, it replaces the date branch by redefinition — the original file is never modified.

Single point of change

Any future platform date-type change is handled here, not across every date recipe.

POST-IMPLEMENTATION EFFECT

- ✓ Recency variables return days since the event, matching legacy behaviour
- ✓ Every date variable carries a consistent, verifiable offset — no silent drift
- ✓ Legacy comparison became possible: both sides convert to calendar dates from a known basis
- ✓ The build-date gap between platforms is now a documented adjustment

WHAT BOTH BRIDGES HAVE IN COMMON

Neither changes a line of the original files. Both load after the code they adjust and override by redefinition — the legacy engine stays intact and every CDP accommodation is visible in one place

FINDING · BUILD DATE

Recent Build Dates Populate More Than Older Ones

A late observation that changed the remaining defect list. The build date is not just a cut-off for date arithmetic — it decides how much of the CDP's history a run can actually reach

WHY THE BUILD DATE CHANGES THE OUTCOME

Every filter is anchored to it

Each recipe looks backwards from the build date. An older date excludes everything loaded since.

The CDP is still being populated

Migration and daily feeds keep landing. A run dated before a load cannot see rows that now exist.

Recency variables are most sensitive

One- and two-year windows sit closest to the boundary and gain most from a newer date.

It is not a code path

Nothing in the engine changes — the same recipes simply reach more history.

WHAT WE OBSERVED · THREE STAGES

OLDER BUILD DATE	RECENT BUILD DATE	AFTER FINAL CLOSURE
464 / 475	469 / 475	475 / 475
97.7% populating	98.7% populating	100% populating

- **Five variables populated with no code change**

  The same build reached data loaded after the earlier date.

- **It narrowed the escalation list**

  Part of an apparent data gap was a build-date artefact — so only six genuine items went to the final iteration.

- **Then the last six were closed deliberately**

  Each with its own root cause and business sign-off — next slide.

WHAT THIS CHANGES ABOUT READING A RESULT

A variable reported empty is only empty relative to the build date it ran against. Before escalating a gap as missing data, confirm the run can reach the period the data would sit in

CLOSURE · THE LAST SIX VARIABLES

From 469 to 475: How the Final Six Were Closed

The last six looked like data gaps and design limits. Each was taken back through the derivation path, given a reasoned judgement, agreed with the business — and closed. None was waived

1 · Channel variable

CODE + BUSINESS RULE

SYMPTOM

Non-web shipment count still zero after the build-date move.

ROOT CAUSE

Non-web orders carry a blank channel code on the CDP. The not-equal filter sat in a sign slot the first NULL-safe fix never reached, so every blank row was dropped.

JUDGEMENT & RESOLUTION

NULL-safe comparison extended to all sign slots; client confirmed a blank channel code means non-web; paid-only filter added and signed off.

✓ **Probe: blank code on >99.9% of paid non-web orders**

2 · Promotion variables

REFERENCE DATA

SYMPTOM

Two title-promotion counts zero for every customer.

ROOT CAUSE

The recipes filter on hard-coded legacy product identifiers that were retired in migration — only ~10% of that title family survived in the CDP cross-reference.

JUDGEMENT & RESOLUTION

Client supplied the current identifiers; a legacy→current lookup was added to the bridge, so the recipe text stayed unchanged.

✓ **Match on refreshed keys reconciled to legacy within tolerance**

3 · Activity-sparse variables

VALIDATION SCOPE

SYMPTOM

Club count, returner flag and one unported variable empty.

ROOT CAUSE

Two were correct but empty on the test harness — its customers had no recent club orders and no returns above threshold. The third had never been ported.

JUDGEMENT & RESOLUTION

The two were validated on the full ~600K audience, where they populate and match legacy; the missing variable was ported as one new recipe on an existing builder.

✓ **Full-audience census: 475 of 475 populating**

THE PRINCIPLE BEHIND THE LAST MILE

A gap is only accepted when every route to close it has been tried. Here, a sign-slot sweep, a reference refresh and a validation on the right population turned three 'won't fix' items into 100%

What the Code Could Not Decide

Several fixes were technically clear but changed what a number means. Those were never shipped on engineering judgement alone — each went to the business owner with evidence and was recorded in a decision register

	THE ISSUE	HOW IT WAS GOVERNED
Two-field date logic	Use the original created date where present, otherwise the platform load timestamp.	Agreed with the platform build team and the client's modelling lead before implementation.
Blank channel = non-web	Almost every non-web order arrived with a blank channel code.	Raised as an assumption with evidence; confirmed by the client before the final run.
Paid-only shipment counts	A shipment variable described as 'paid' counted unpaid orders — ~7% too high.	Lowers a figure the client sees, so released only after business sign-off.
Retired product identifiers	Hard-coded legacy identifiers no longer existed in the CDP.	Client supplied current identifiers; mapping held in the bridge, recipe untouched.
New reference codes	A product code appeared in the CDP that lookups had never seen.	Parked in a neutral group, then moved to its family once the client named it.

THE RULE WE WORKED TO

Engineering decides whether the code does what it says. The business decides what it should say. Any fix that changes a number's meaning needs both — and a written record of why

Twenty Lessons From the Migration — 01 to 10

Diagnosis and working with AI. Each lesson has its own slide, with the moment that produced it — tagged by who surfaced it: the team, the model, or both together

01 AI + TEAM

Give it evidence, not adjectives

Counts, log lines and profiles turn a guess into a diagnosis.

02 TEAM FOUND

Push back on the first answer

The first explanation was coherent, confident and wrong — twice.

03 AI FOUND

Diagnose at the source, not the output

A blank column says nothing about why it is blank.

04 AI FOUND

Fix the generator, not the symptom

One shared predicate served 118 variables.

05 AI + TEAM

A clean log is not a correct output

Zero errors and 31 empty variables coexisted.

06 AI + TEAM

Separate 'missing' from 'wrong'

Absent, zero-filled and understated are different faults.

07 TEAM FOUND

Invest early in a feedback loop

Verification, not repair, was the bottleneck.

08 AI + TEAM

Know when it is data — then close it

Data gaps closed only when escalated with evidence.

09 AI FOUND

Read the log the run actually wrote

The visible log showed the symptom; the cause was redirected.

10 AI + TEAM

Guard shared state before every run

A leftover table produced a 'successful' empty run.

THE THREAD THROUGH 01–10

Verify against the source, fix at the level the defect lives, and prove the result by value

Twenty Lessons From the Migration — 11 to 20

Delivery, safety and governance on a shared production platform — the lessons that kept fast work from turning into risky work

11 — TEAM FOUND

Edit the real file, prove it compiled

One overwritten file erased every bridge macro.

12 — TEAM FOUND

Keep the client's code theirs

Bridges absorbed the platform; recipes stayed intact.

13 — AI FOUND

Check key domains before merging

Two look-alike keys turned a join into an append.

14 — AI + TEAM

Normalise before you compare

Different build dates make raw values incomparable.

15 — AI FOUND

Measure where the time goes

Query count, not audience size, set the clock.

16 — TEAM FOUND

The business owns what a number means

Meaning-changing fixes needed sign-off.

17 — AI + TEAM

Expect reference data to drift

New and retired codes appeared after go-live.

18 — AI + TEAM

Test the fix, not just the variable

Two fixes introduced defects of their own.

19 — AI FOUND

Sweep the whole defect class

The NULL bug lived in more slots than the first fix reached.

20 — TEAM FOUND

Write it down as you go

Registers turned decisions into evidence.

THE THREAD THROUGH 11–20

Protect production, keep the client's code theirs, and let the business own meaning — then write every decision down

LESSON 01 OF 20

AI + TEAM

01

Give It Evidence, Not Adjectives

"The variable looks wrong" produces a guess. The log line, the row counts and the column profile produce a diagnosis.

Every hour spent gathering evidence before asking saved several spent chasing a plausible answer that happened to be wrong.

WHAT THIS LOOKED LIKE IN PRACTICE

Adjective

"Magazine revenue is coming out too low." → A list of things it could conceivably be, none of them verified.

Evidence

The emitted predicate, the distribution of the order-type code, and the revenue sum computed both ways against source.

Result

The NULL-dropping defect was identified and sized in one pass — roughly a third of sampled magazine revenue.

THE ASYMMETRY WORTH REMEMBERING

A vague prompt returns a fluent answer that is expensive to disprove. A prompt carrying counts, log lines and column profiles returns something you can act on — or reject — immediately.

The evidence gathered to brief the model is also the evidence the client sees in the update.

Bring the artefact, not the impression

LESSON 02 OF 20

TEAM FOUND

02

Push Back on the First Answer

The first explanation is usually coherent, confident and occasionally wrong. Here it was wrong twice — and both times the fix it implied would have been substantial.

The habit that caught it: ask what evidence would disprove this, then go and look.

TWO TIMES THE FIRST ANSWER DID NOT SURVIVE CONTACT

First answer

A product code spans titles with different content classes, so the affinity join resolves ambiguously — build a title-level lookup.

What profiling found

The mapped column held no values at all. The join could not match anything, ambiguously or otherwise.

Second answer

Switch the title join to a different key. Testing showed zero matches against tens of millions for the existing key — it would have broken 25 working variables.

WHAT THE PUSHBACK WAS WORTH

A lookup table, a parse rule and a key change were replaced by one-line corrections — and a change that would have caused a regression was stopped before it shipped.

The cost of asking was one read-only query; the cost of not asking would have been weeks.

Confidence is not evidence — verify, then commit

LESSON 03 OF 20

AI FOUND

03

Diagnose at Source, Not in the Output

A blank column tells you something failed. It tells you nothing about why.

The answer was rarely in the code — it was in the CDP column the code pointed at, and it usually took one read-only query to find.

THE THREE QUESTIONS ASKED OF EVERY SUSPECT COLUMN

Is the column populated?

The mapped product-code column was NULL on every row. Every join against it compared NULL to everything.

Do the join keys share a domain?

Two customer keys looked interchangeable and were not — every single value differed.

Do the filter values exist?

The channel code was blank on almost every non-web order, so a filter written as 'not web' could never match them.

WHAT THIS CHANGED

Three speculative rebuilds were replaced by targeted corrections, because profiling showed each fault sat upstream of the logic entirely.

The output shows the symptom; only the source shows the cause

LESSON 04 OF 20

AI FOUND

04

Fix the Generator, Not the Symptom

In a macro-driven engine the code you read is not the code that runs. SQL is written at execution time by shared builders.

Patching a variable treats one instance of a pattern. Correcting the builder treats every instance at once.

ONE PREDICATE, 118 VARIABLES

The defect

A bare inequality in the shared exclusion branch dropped every NULL row under ANSI three-valued logic.

The reach

That one branch sits inside three query builders and serves 118 requested variables.

The correction

Six lines changed — two per builder. The ~5K-error cascade cleared, 13 variables recovered and values corrected across the rest.

THE INVERSE ALSO HOLDS

A defect introduced at generator level multiplies just as efficiently. An interim fix worked on text columns and failed on numeric ones, taking out 13 variables until it was replaced with a type-agnostic test.

Leverage cuts both ways — find the level the defect lives at

LESSON 05 OF 20

AI + TEAM

05

A Clean Log Is Not a Correct Output

Zero errors means the code executed. It says nothing about whether the numbers are right.

The most expensive defect on this project produced no error at all — it quietly dropped rows and understated revenue by about a third.

WHAT A ZERO-ERROR RUN STILL CONTAINED

31 variables at zero

Silently empty for every customer in the test audience. The log recorded no failure of any kind.

Revenue understated

NULL rows dropped by an inequality filter. No warning, no message — just a smaller number than legacy.

4 variables entirely blank

Present in the structure, never populated. Only a column-level census surfaced them.

WHAT REPLACED THE LOG AS THE GATE

A variable-level population census — every column classified as populating, all-zero or all-blank — became the acceptance artefact. The zero list is the defect queue; the log only confirms the run finished.

Absence of failure is not evidence of correctness

LESSON 06 OF 20

AI + TEAM

06

Separate 'Missing' From 'Wrong'

Three families were reported the same way: "it isn't right". They had nothing in common.

Conflating the categories sends you looking for the wrong evidence — and the wrong test returns a confident, useless answer.

THREE FAULTS THAT LOOK ALIKE AND ARE NOT

Absent

The variable never built: the query failed, the staging table was never created, the column was dropped. Look in the log.

Zero-filled

The variable built but matched nothing. The join or filter is at fault. Profile the source column.

Understated

The variable built and matched, but rows were silently excluded. Only a value comparison reveals it.

WHY THE DISTINCTION PAID

The interest variables were absent, the affinity variables were zero-filled, and the revenue variables were understated. Three categories, three tests, three unrelated root causes — and one shared reporting phrase.

Classify the symptom before you choose the test

LESSON 07 OF 20

TEAM FOUND

07

Invest Early in a Feedback Loop

A full run took about an hour. That made verification, not repair, the constraint on the whole engagement.

A small audience that reproduces the full failure profile pays for itself within a day — and it should be built in week one.

WHAT THE HARNESS HAD TO GET RIGHT

Reproduce the failures

Same ~5K errors, same type-failure cascade, same 13 missing variables as the full production run.

Select on the right measure

Customers ranked by variables actually carrying data — the engine zero-fills unmatched numerics, so non-missing is not populated.

Stay isolated

Registered under its own keycode and output name so production was never touched while debugging.

AND KNOW WHAT IT CANNOT TELL YOU

A harness of the richest records proves code paths and misstates fill rates. Three of the final six variables looked empty on the harness and populated on the full audience — final population is always measured on the real population.

Optimise the loop you run most, not the one you run once

LESSON 08 OF 20

AI + TEAM

08

Know When It Is Data — Then Close It

Some gaps cannot be fixed in code alone, because the data arrived differently or not at all.

Recognising that early turns an open defect into an evidenced ask — and a well-framed ask gets answered.

HOW DATA GAPS WERE CLOSED, NOT WAIVED

Profile and evidence

Blank channel codes on >99.9% of non-web orders; retired product identifiers absent from the cross-reference.

Confirm against legacy

The client verified the same variables populate on the legacy platform — the data exists, it just arrived differently.

Ask with a proposal

Each gap went to the business with a proposed rule or mapping. Both were confirmed, implemented in the bridge, and closed.

THE OTHER HALF OF THIS LESSON

Rule out the run before blaming the data. Five apparent gaps closed simply by using a recent build date, and three more were only empty on the test sample. Escalate what remains — never what you have not yet checked.

Not every gap is a bug — prove which kind, then close it

LESSON 09 OF 20

AI FOUND

09

Read the Log the Run Actually Wrote

The engine redirects its log part-way through a run. The log on screen ended with a generic 'file does not exist' error.

The real cause — an integrity-check failure — sat in a different file that most people never opened.

WHAT THE TWO LOGS SAID

Visible log

Two errors: the output dataset did not exist. Accurate, and useless — it described the consequence, not the cause.

Redirected log

An integrity check had failed on an unseen product code, so the run stopped before building a single variable.

Side effect

The previous good output had been renamed to a backup — another failed run would have overwritten it.

THE HABIT THAT FIXED IT

Every diagnosis now starts by locating the run's own log and searching for the first error, not the last. The last error is almost always downstream noise; the first is almost always the cause.

The first error is the cause; the last is the noise

LESSON 10 OF 20

AI + TEAM

10

Guard Shared State Before Every Run

The engine creates a permanent audience table in a shared schema and drops it only on clean completion.

One aborted run left it behind — and the next run failed silently, producing a 'successful' empty output.

HOW A LEFTOVER TABLE BECAME A SILENT FAILURE

Aborted run

The run stopped mid-way; the clean-up step that drops the audience table never executed.

Next run

CREATE failed because the table already existed; the audience loaded as nothing; every variable joined to an empty set.

Guard added

An existence check drops any stale table before rebuild — on every run, automatically.

THE GENERAL RULE

Any object that outlives a run — permanent tables, renamed backups, cached lookups — is shared state. Assume a previous run failed halfway, and make every run safe to start from that condition.

Assume the last run died halfway

LESSON 11 OF 20

TEAM FOUND

11

Edit the Real File, Then Prove It Compiled

A one-macro patch was saved over the entire compatibility file. Every mapping macro vanished.

The run did not fail loudly — it produced thousands of 'macro not resolved' warnings and lost hundreds of variables.

WHAT CHANGED IN HOW FIXES WERE SHIPPED

Edit, don't reconstruct

Fixes applied byte-for-byte to the uploaded source — encoding and line endings preserved — never retyped from memory.

Diff every change

Each delivered file diffed against the original, so only the intended lines could differ.

Compile check

After deploy, a fresh session confirms every macro the file should define actually exists — before the hour-long run.

WHY A FRESH SESSION

A session that already loaded the old file will still 'have' the macros and pass the check falsely. The check only counts if it runs where nothing was loaded before.

Ship the file you diffed, then prove it loaded

LESSON 12 OF 20

TEAM FOUND

12

Keep the Client's Code Recognisably Theirs

The fastest route to working output was to edit 500+ recipe calls directly. It would also have left the client with code they no longer recognised.

Two bridge files absorbed the platform instead.

WHAT THE BRIDGE PATTERN PROTECTED

Ownership

The client's analysts can still read, maintain and extend the recipes they wrote — nothing in them refers to the CDP.

Reviewability

Every platform accommodation lives in two files. A reviewer reads two files, not a diff spread across thousands of lines.

Reversibility

A wrong mapping is a one-line correction in one place, and it fixes every variable that depends on it.

THE DISCIPLINE IT REQUIRED

Override by redefinition, loaded after the original. The recipes were touched only where the business asked for a change of meaning — never to accommodate the platform.

Translate at the boundary, not inside the logic

LESSON 13 OF 20

AI FOUND

13

Check Key Domains Before You Merge

Two customer identifiers had similar names and similar-looking values. A new recipe grouped on one and labelled it as the other.

The merge matched nothing — and appended every row instead.

WHAT A KEY MISMATCH LOOKS LIKE

Symptom

Output grew by ~50%: the new rows carried the new variable and nothing else.

Diagnosis

A side-by-side of both keys showed every single value differed — two separate domains, not two names for one.

Fix

Join on the platform key, but select and group on the customer key, so the output stays in the merge domain.

THE GATE THAT CATCHES IT

Output row count must equal the requested audience exactly — every run. A key-domain error almost always shows up as rows that should not exist, long before anyone looks at values.

Same name is not same domain — test the overlap

LESSON 14 OF 20

AI + TEAM

14

Normalise Before You Compare

Legacy and CDP outputs were built a year apart and store dates as days since build.

Compared raw, almost every date variable looks wrong. Compared correctly, almost all of them match.

WHAT HAD TO BE NORMALISED

Build date

Both sides converted to calendar dates from their own build date before any comparison.

Universe

Matched only on customers present in both populations; non-overlapping customers excluded rather than reported as mismatches.

Expected drift

Extra months of activity and re-classified products banded as expected variance, not defects.

WHY THIS IS A LESSON, NOT A FOOTNOTE

An un-normalised comparison would have produced dozens of false defects — each one costing a diagnosis. Getting the comparison right first is what made the defect list trustworthy.

Compare like with like, or every number lies

LESSON 15 OF 20

AI FOUND

15

Measure Where the Time Goes

Shrinking the audience 570-fold barely changed the runtime. Intuition said otherwise.

Timing every query in the log showed the cost was the number of queries, not the size of the data.

WHAT THE TIMING SHOWED

Fixed floor

~5 seconds per query regardless of rows — scheduling and round-trip latency, not data volume.

Driver

~2,400 queries per full run. Runtime scales with variables requested, not customers.

Right lever

Subset the variable list when debugging; parallelise families and stage heavy views for production.

WHAT IT SAVED

Effort that would have gone into optimising audience size went into the levers that move the clock. Debug runs dropped from an hour to minutes once only the affected variables were requested.

Measure before you optimise

LESSON 16 OF 20

TEAM FOUND

16

The Business Owns What a Number Means

Some fixes were technically obvious and still not ours to ship alone — because they changed what a number meant to the people using it.

Engineering proved the effect; the business decided.

FIXES THAT NEEDED A BUSINESS DECISION

Paid-only counts

Correct per the variable's own definition — and it lowered a figure the client relied on by ~7%.

Blank = non-web

A sound inference from the data, but an assumption about meaning until the client confirmed it.

Two-field date

Agreed jointly with the platform team and modelling lead before it reached any model.

HOW IT WAS RUN

Each decision went to the owner with the evidence, the before/after impact, and a recommendation — then into the decision register with the date and the approver.

Prove the effect; let the owner decide the meaning

LESSON 17 OF 20

AI + TEAM

17

Expect Reference Data to Drift

The code assumed a fixed set of product codes. The live CDP kept moving: new codes appeared after go-live, and some legacy codes were retired.

Both broke things — in opposite directions.

TWO KINDS OF DRIFT

New codes

An unseen code tripped the integrity check and stopped the whole run. Parked in a neutral group until the client named its family.

Retired codes

Hard-coded legacy identifiers no longer existed, so two variables silently matched nothing. Refreshed via a client-supplied lookup.

Standing check

Reference tables are now compared against the live CDP before each production run.

THE GENERAL RULE

Hard-coded lists are a time bomb on a live platform. Keep them in one lookup, check them against the source each run, and make unknowns fail loudly rather than quietly.

Hard-coded lists drift — check them every run

LESSON 18 OF 20

AI + TEAM

18

Test the Fix, Not Just the Variable

Two fixes on this project introduced defects of their own.

One worked on text columns and failed on numeric ones. Another used an operator word inside macro logic, where the macro language read it as an operator.

WHEN THE FIX WAS THE BUG

Type-unsafe fix

A text default in a NULL guard failed on numeric columns — 13 variables disappeared until a type-agnostic test replaced it.

Macro parsing

A bare operator word inside a macro condition was parsed as an operator, not text — caught before release by quoting it.

Response

Every generator change now runs on the harness against both column types before promotion.

WHY GENERATOR FIXES NEED MORE TESTING

A generator-level fix touches every variable that routes through it. Its blast radius is its whole reach — so it is tested across types and operators, not just on the variable that prompted it.

A fix in a shared builder is a release, not a patch

LESSON 19 OF 20

AI FOUND

19

Sweep the Whole Defect Class

The first NULL-safe fix covered the exclusion slots and cleared the error cascade. It did not cover the sign slots, where the same bare not-equal lived.

The last channel variable stayed at zero because of it.

FINDING EVERY PLACE THE DEFECT CAN LIVE

Name the class

NULL compared with not-equal evaluates to UNKNOWN and drops the row — wherever it is emitted.

Enumerate the sites

Exclusion slots, sign slots, every builder, every not-equal spelling — and every recipe routed through them.

Close it once

One sweep across all sites, with a list of every variable whose values could move.

WHAT A SWEEP PRODUCES

Not just a fix, but an impact list: which variables changed status, which changed value only, and which could not move. That list is what the client reviews.

Fix the class, then list everything it touched

LESSON 20 OF 20

TEAM FOUND

20

Write It Down as You Go

Three months, seven iterations, dozens of fixes and judgement calls. Without a record, every question about 'why is this number different' becomes an investigation.

With one, it is a lookup.

WHAT WAS RECORDED, AND WHEN

Fix register

Every change: file, lines, root cause, evidence, verification run and result.

Decision register

Every meaning change: the issue, evidence, recommendation, approver and date.

Architecture & lineage

The map the system never had — kept current as bridges and recipes changed.

WHY IT PAID BACK

Stakeholder updates, legacy reconciliations and this case study were written from the registers, not from memory. The documentation was the cheapest part of the project — because it was never done after the fact.

If it is not written down, it did not happen

Eight Takeaways for the Next Legacy Migration

What we would carry into any similar engagement — and what each one changes about how we scope, staff and sequence the work

01 Fund discovery as a phase

Undocumented estates need a paid comprehension phase before any migration estimate is credible.

Changes how we estimate

02 Absorb differences in a bridge layer

Platform accommodations in two files; original code untouched and reviewable.

Our default pattern

03 Audit NULL & type semantics on day one

Every not-equal filter and date conversion checked against the target platform before the first run.

Reusable pre-flight check

04 Make the census the acceptance gate

Contract against variable population and reconciliation, not error counts.

Redefines 'done'

05 Build harness & guards in week one

Small audience, isolated outputs, stale-table guards, post-deploy compile checks.

Protects production

06 Close gaps — don't waive them

Rule out build date and sample scope, then escalate with evidence and a proposal.

Path to 100%

07 Route meaning changes to the business

Evidence, impact and recommendation to the owner; decisions logged.

Builds client trust

08 Size runtime by queries, not rows

Plan parallelism and staging around query count and per-query latency.

Accurate run windows

WHY THIS MATTERS BEYOND ONE CLIENT

Any organisation running undocumented legacy analytics faces the same problems — no lineage, changed platform semantics, and no way to prove correctness. This method answers all three, to 100% population

WHERE THIS LEAVES US

Discovery Was the Deliverable

The engagement was scoped as a remapping exercise. What it required was reconstructing an undocumented system well enough to reason about it — and that understanding, not any single fix, is what took the pipeline from ~5,000 errors to 475 of 475 variables populating.

A pipeline that runs

Zero errors, 100% of 475 variables populating, reconciled against legacy and signed off.

A codebase still the client's

Original library untouched; two bridges hold every platform accommodation, documented and reviewable.

A repeatable method

Diagnose at source, fix the generator, prove by value, govern meaning with the business, write it all down.

No variable was waived. Every closure — code, reference data or business rule — is evidenced at source and recorded in the fix and decision registers